Applied Cryptography
Lect. #2

Marco Calderini

University of Trento

A.Y. 2024/2025

Last time:

- perfect secrecy
- OTP has perfect secrecy
- No cipher with $|\mathcal{P}| > |\mathcal{K}|$ can be perfect

This time:

- semantic security
- stream ciphers: PRG
- LFSR

From Perfect secrecy to Semantic security

Issues with OTP

- How to generate perfectly uniform random keys?
- How to distribute a large amount of key material as much as the data that we want to send?

Perfect secrecy is too much

Definition (Perfect secrecy alt.)

(Enc, Dec) cipher with uniform random keys. $\forall m_0, m_1 \in \mathcal{P}$ and $\forall c \in \mathcal{C}$:

$$Pr[Enc_K(m_0) = c] = Pr[Enc_K(m_1) = c].$$

I.e. $Enc_K(m_0)$ $Enc_K(m_1)$ are identically distributed.

Perfect secrecy requires that absolutely **no information** about an encrypted message is leaked, even to an eavesdropper with **unlimited** computational power.

How to relax perfect secrecy?

Idea:

- Security is only guaranteed against efficient adversaries that run for some feasible amount of time.

  This means that given enough time (or sufficient computational resources) an attacker may be able to violate security. If we can make the resources required to break the scheme larger than those available to any realistic attacker, then for all practical purposes the scheme is unbreakable.
- Adversaries can potentially succeed (i.e., security can potentially fail) with some very small probability. If we can make this probability sufficiently small, we need not worry about it.
- Introduce a game-style security definition

Big-O notation

The time complexity of algorithms can be expressed as a function of the size of the input that expresses the number of elementary operations performed

The Big O notation is a convenient way to describe how fast a function, describing the time complexity of an algorithm, is growing

Definition

Let $f(n)$ and $g(n)$ be two positive functions. We say that $g(n)$ has order of $f(n)$,in symbols $g(n) \in O(f(n))$, if there are positive constants M and n_0 such that $g(n) \leq M * f(n)$ for all $n \geq n_0$.

Intuition: $g(n) \in O(f(n))$ means that $g(n)$ does not grow faster than $f(n)$

- When $g(n) \in O(1)$ we say that the algorithm has **constant** time complexity
- When $g(n) \in O(\log(n))$ we say that the algorithm has **logarithmic** time complexity
- When $g(n) \in O(n)$ we say that the algorithm has **linear** time complexity
- When $g(n) \in O(n \log(n))$ we say that the algorithm has **linearithmic** time complexity
- When $g(n) \in O(n^2)$ we say that the algorithm has **quadratic** time complexity
- When $g(n) \in O(n^k)$we say that the algorithm has **polynomial** time complexity for $k > 2$
- When $g(n) \in O(e^n)$ we say that the algorithm has **exponential** time complexity for c reasonable a constant greater than 2
- When $g(n) \in O(n!)$ we say that the algorithm has **factorial** time complexity

Efficient adversary

When honest parties initialize a scheme (i.e., when they generate keys) they choose some value n for the security parameter.

Example

A security parameter is: the bit-length of the key/message.

We equate *efficient adversaries* with randomized (i.e., probabilistic) algorithms running in time polynomial in n (PPT). This means there is some polynomial p such that the adversary runs for time at most $p(n)$

Note: we allow all algorithms to be probabilistic (or randomized). Any such algorithm may "toss a coin" at each step of its execution; that is, the algorithm can access an unbiased random bit at each step.

Practical viewpoint: Adversary has 2^{100} running time

Negligible, super-poly, and poly-bounded functions

Definition

A function $f : \mathbb{Z}_{\geq 1} \to \mathbb{R}$ is called **negligible** if for all $c \in \mathbb{N}$, $c > 0$, there exists $n_0 \in \mathbb{Z}_{\geq 1}$ such that for all integers $n \geq n_0$, we have $|f(n)| < 1/n^c$.

Equivalently: for all $c > 0$, we have $\lim_{n\to\infty} f(n)n^c = 0$.

Example

Negligible functions: 2^{-n}, $2^{-\sqrt{n}}$, $n^{-\log n}$.

Definition

A function $f : \mathbb{Z}_{\geq 1} \to \mathbb{R}$ is called super-poly if $1/f$ is negligible.

Definition

A function $f : \mathbb{Z}_{\geq 1} \to \mathbb{R}$ is called poly-bounded, if there exists $c, d > 0$ s.t. $|f(n)| \leq n^c + d$.

Fact

If ε and ε' are negligible values, and Q and Q' are poly-bounded values, then:

(i) $\varepsilon + \varepsilon'$ is a negligible value,

(ii) $Q + Q'$ and $Q \cdot Q'$ are poly-bounded,

(iii) $Q \cdot \varepsilon$ is a negligible value.

Attack games

An attack game played between two parties, a challenger and an adversary.

- The challenger and the adversary send messages back and forth to each other
- at the end of the game, the adversary may output some value
- The attack game also defines a probability space, and this in turn defines the adversary's advantage

Semantic security

Attack Game (SS)

For a given cipher $\mathcal{E} = (Enc, Dec)$, defined over $(\mathcal{K}, \mathcal{P}, \mathcal{C})$, and for a given adversary $\mathcal{A}$, we define two experiments, Experiment 0 and Experiment 1. For $b = 0, 1$, we define

Experiment b:

- The adversary computes $m_0, m_1 \in \mathcal{P}$, of the <u>same length</u>, and sends them to the challenger.
- The challenger computes $k \xleftarrow{R} \mathcal{K}, c \xleftarrow{R} Enc(k, m_b)$, and sends c to the adversary.
- The adversary outputs a bit $\hat{b} \in \{0, 1\}$.

Definition

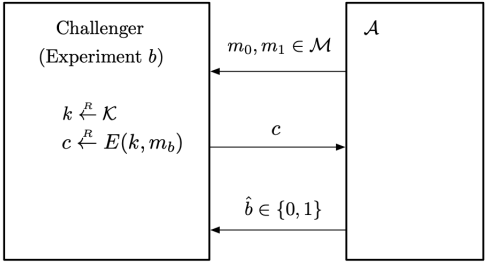
For $b = 0, 1$, let W_b be the event that $\mathcal{A}$ outputs 1 in Experiment b. We define $\mathcal{A}$'s **semantic security advantage** with respect to $\mathcal{E}$ as

$$SSadv[A, E] := |Pr[W_0] - Pr[W_1]|$$

.

Semantic security

Experiment b of the SS game



Semantic security

Definition

A cipher $\mathcal{E}$ is semantically secure if for all efficient adversaries $\mathcal{A}$, the value $SSadv[A, E]$ is negligible.

Bit-guessing version of the attack game

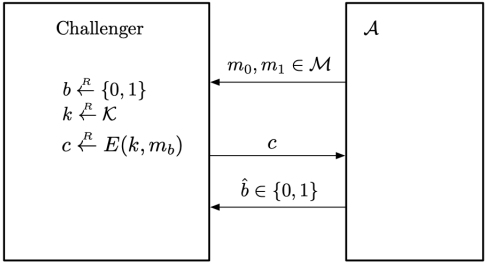
Attack Game (bit-guessing SS):

For a given cipher $\mathcal{E} = (Enc, Dec)$, defined over $(\mathcal{K}, \mathcal{P}, \mathcal{C})$, and for a given adversary $\mathcal{A}$, the attack game runs as follows:

- The adversary computes $m_0, m_1 \in \mathcal{P}$, of the <u>same length</u>, and sends them to the challenger.
- The challenger computes $b \xleftarrow{R} \{0,1\}$, $k \xleftarrow{R} \mathcal{K}$, $c \xleftarrow{R} Enc(k, m_b)$, and sends c to the adversary.
- The adversary outputs a bit $\hat{b} \in \{0,1\}$.

We say that $\mathcal{A}$ wins the game if $\hat{b} = b$.

Bit-guessing version of the attack game



Bit-guessing version of the attack game: remarks

- instead of having two different experiments, there is just a single experiment.
- any adversary can win the game with probability $1/2$, simply by ignoring c completely and choosing $\hat{b}$ at random
- we are interested in is how much better than random guessing an adversary can do.
- W event that the adversary wins the bit-guessing version of the attack game, then we are interested in the quantity $|Pr[W] - 1/2| = SSadv^*[\mathcal{A}, \mathcal{E}]$.

Theorem

For every cipher $\mathcal{E}$ and every adversary A*, we have*

$$SSadv[\mathcal{A}, \mathcal{E}] = 2 \cdot SSadv^*[\mathcal{A}, \mathcal{E}].$$

Proof.

Note that $Pr[W_0] = Pr[\hat{b} = 1 | b = 0]$ and $Pr[W_1] = Pr[\hat{b} = 1 | b = 1]$. So in Game bit-guessing SS we have

$$\begin{aligned}
Pr[\hat{b} = b] =& Pr[\hat{b} = b | b = 0] \cdot Pr[b = 0] + Pr[\hat{b} = b | b = 1] \cdot Pr[b = 1] \\
=& Pr[\hat{b} = 0 | b = 0] \cdot \frac{1}{2} + Pr[\hat{b} = 1 | b = 1] \cdot \frac{1}{2} \\
=& \frac{1}{2}(1 - Pr[\hat{b} = 1 | b = 0] + Pr[\hat{b} = 1 | b = 1]) \\
=& \frac{1}{2}(1 - Pr[W_0] + Pr[W_1]).
\end{aligned}$$

Therefore

$$SSad^*[\mathcal{A}, \mathcal{E}] = |Pr[\hat{b} = b] - 1/2| = \frac{1}{2}|Pr[W_0] - Pr[W_1]|.$$

Remarks on Semantic Security

- The messages m_0 and m_1 in the game have same length: an encryption of a message is allowed to leak the length of the message (but nothing else).
- If $\mathcal{E}$ has perfect secrecy then for every adversary $\mathcal{A}$ (efficient or not), we have $SSadv[\mathcal{A}, \mathcal{E}] = 0$.
- Because the definition of semantic security is a bit more forgiving with regard to variable length message spaces, for variable length one-time pad we have also $SSadv[\mathcal{A}, \mathcal{E}] = 0$.

A stronger impossibility result

Theorem

Let $\mathcal{E}$ be a cipher defined over $(\mathbb{K}, \mathcal{P}, \mathcal{C})$. Suppose that $SSadv[A, E] \leq \varepsilon$ for all adversaries $\mathcal{A}$, even including computationally unbounded ones. Then

$$|\mathcal{K}| \geq (1 - \varepsilon)|\mathcal{C}|.$$

Other notions of security

Connections to weaker notions of security

Message recovery attacks:

An adversary is given an encryption of a random message, and is able to recover the message from the ciphertext with probability significantly better than random guessing, that is, probability $1/|\mathcal{P}|$.

Message recovery attacks

Attack Game (MR)

Let cipher $\mathcal{E} = (Enc, Dec)$ be a cipher defined over $(\mathcal{K}, \mathcal{P}, \mathcal{C})$, and for a given adversary $\mathcal{A}$, the attack game proceeds as follows:

- The challenger computes $m \xleftarrow{R} \mathcal{P}, k \xleftarrow{R} \mathcal{K}, c \xleftarrow{R} Enc(k, m)$, and sends c to the adversary.
- The adversary outputs a message $\bar{m} \in \mathcal{P}$.

Let W be the event that $\bar{m} = m$. We say that $\mathcal{A}$ wins the game in this case, and we define $\mathcal{A}$'s message recovery advantage with respect to $\mathcal{E}$ as $MRadv[\mathcal{A}, \mathcal{E}] := |Pr[W] - 1/|\mathcal{P}||$.

Definition

A cipher $\mathcal{E}$ is secure against message recovery if for all efficient adversaries $\mathcal{A}$, the value $MRadv[\mathcal{A}, \mathcal{E}]$ is negligible.

Theorem

Let $\mathcal{E} = (Enc, Dec)$ be a cipher. If $\mathcal{E}$ is semantically secure then $\mathcal{E}$ is secure against message recovery.

Proof.

Assume that $\mathcal{E}$ is not secure against message recovery $\Rightarrow \exists \mathcal{A}$ such that $MRadv[\mathcal{A}, \mathcal{E}]$ is non-negligible.
Idea: construct an attacker $\mathcal{B}$ for Semantic Security that use $\mathcal{A}$ as "black box".
-$\mathcal{B}$ generates m_0 and m_1 for its SS challenger, and receive c from the SS challenger.
-$\mathcal{B}$ sends c to $\mathcal{A}$ (as if it were coming from $\mathcal{A}$'s MR challenger)
-$\mathcal{A}$ outputs to $\mathcal{B}$ with $\bar{m}$.
-if $\bar{m} = m_1$ then $\mathcal{B}$ outputs $\hat{b} = 1$, and outputs $\hat{b} = 0$ otherwise.
Then

$$SSadv[\mathcal{B}, \mathcal{E}] \geq MRadv[\mathcal{A}, \mathcal{E}].$$

$\square$

Computing individual bits of a message

Let $m \in \mathcal{P}$, we define $parity(m)$ is the exclusive-OR (XOR) of all the individual bits of m.

Attack Game (parity prediction)

Given cipher $\mathcal{E} = (Enc, Dec)$ and adversary $\mathcal{A}$, the attack game proceeds as follows:

- The challenger computes $m \xleftarrow{R} \mathcal{P}, k \xleftarrow{R} \mathcal{K}, c \xleftarrow{R} Enc(k, m)$, and sends c to the adversary.
- The adversary outputs $\hat{b} \in \{0, 1\}$.

Let W be the event that $\hat{b} = parity(m)$. We define $\mathcal{A}$'s parity prediction advantage as $Parityadv[\mathcal{A}, \mathcal{E}] := |Pr[W] - 1/2|$.

Definition

A cipher $\mathcal{E}$ is secure against parity prediction if for all efficient adversaries $\mathcal{A}$, the value $Parityadv[\mathcal{A}, \mathcal{E}]$ is negligible.

Theorem

If $\mathcal{E}$ is semantically secure, then $\mathcal{E}$ is secure against parity prediction.

Proof.

Using the same approach as above. Let $\mathcal{A}$ an adversary for the parity prediction with advantage $Parityadv[\mathcal{A}, \mathcal{E}]$. We can create an adversary $\mathcal{B}$ for the bit-guessing version of SS game.

-$\mathcal{B}$ generates m_0 and take $m = 1$ equal to m_0 with one bit flipped for its SS challenger, and receive c from the SS challenger.
-$\mathcal{B}$ sends c to $\mathcal{A}$
-$\mathcal{A}$ outputs to $\mathcal{B}$ with $\hat{b} = parity(m_b)$.
-if $\hat{b} = parity(m_1)$ then $\mathcal{B}$ outputs 1, and outputs 0 otherwise.

Then $SSadv^*[\mathcal{B}, \mathcal{E}] = Parityadv[\mathcal{A}, \mathcal{E}]$. $\square$

Stream ciphers

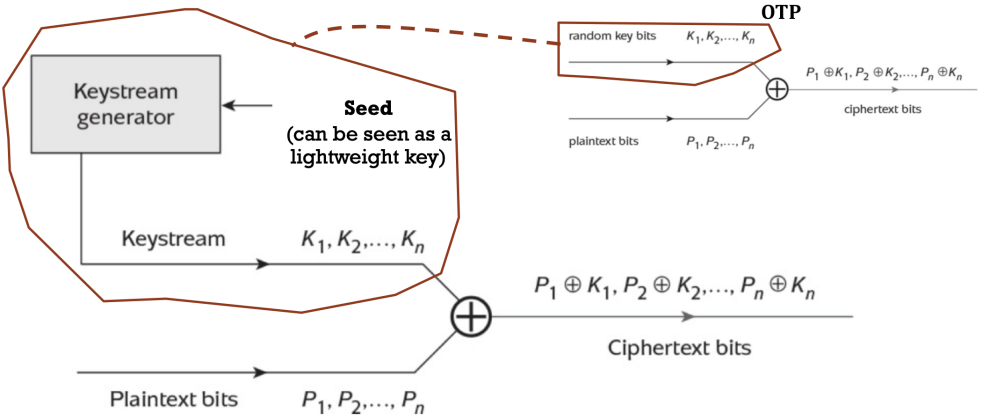
Relaxing the one-time pad

Recall

$$Enc_k(m) = k \oplus m,\ Dec_k(c) = c \oplus k$$

Instead of requiring k to be truly random, replace k with some pseudorandom values generated by an algorithm G from a much shorter seed s.

Stream Cipher: idea



Q: Can a stream cipher have perfect secrecy?

Q: Can a stream cipher have perfect secrecy?

A: No, since the key is shorter than the message

Stream ciphers cannot have perfect secrecy !!

Pseudorandom Generator

A pseudo-random generator (PRG), is an efficient, deterministic algorithm G that, given as input a seed $s \in \{0,1\}^\ell$, computes an output $r \in \{0,1\}^L$ ($\ell < L$).

We want that if s is chosen at random from $\{0,1\}^\ell$ and r is chosen at random from $\{0,1\}^L$, then no efficient adversary can effectively tell the difference between $G(s)$ and r: the two are **computationally indistinguishable**.

Attack Game (PRG)

Given a PRG G and an adversary $\mathcal{A}$, we define two experiments, Experiment 0 and Experiment 1. For $b = 0, 1$ the challenger computes $r \in \{0,1\}^L$ as follow:

- if $b = 0$: $s \xleftarrow{R} \{0,1\}^\ell$, $r = G(s)$,
- if $b = 1$: $r \xleftarrow{R} \{0,1\}^L$.

The challenger sends r to $\mathcal{A}$, which outputs $\bar{b} \in \{0,1\}$.

For $b = 0, 1$, let W_b be the event that $\mathcal{A}$ outputs 1 in Experiment b. We define $\mathcal{A}$'s advantage with respect to G as $PRGadv[\mathcal{A}, G] = |Pr[W_1] - Pr[W_0]|$.

Definition

A PRG G is secure if the value $PRGadv[\mathcal{A}, G]$ is negligible for all efficient adversaries A.

Statistical tests

How could you tell if a string was random or not?

Idea: Run some algorithm on the string and check if expected properties differ much from expectation.

- Count number of 1s
- Look at numbers of long runs
- Analyze sequences of consecutive digits
- ...

Encryption with a PRG

Let G be a PRG, $G : \{0,1\}^\ell \to \{0,1\}^L$. The **stream cipher** $\mathcal{E} = (Enc, Dec)$ constructed from G is defined over $\mathcal{K} = \{0,1\}^\ell$, $\mathcal{P} = \mathcal{C} = \{0,1\}^{\leq L}$ and for $m \in \{0,1\}^{\leq L}$ we have

$$c = Enc(s, m) = G(s)[0, ..., v-1] \oplus m, \; m = Dec(s, c) = G(s)[0, ..., v-1] \oplus c,$$

where $v = |m| = |c|$.

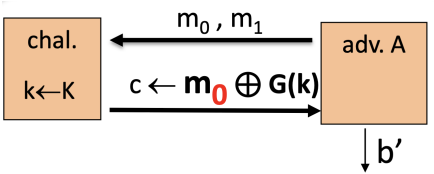
Theorem

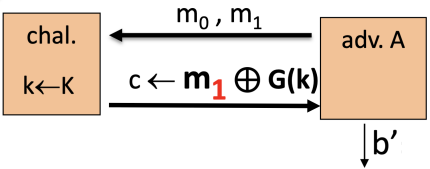
If G is a secure PRG, then the stream cipher $\mathcal{E}$ constructed from G is a semantically secure cipher. In particular, for any sem. sec. adversary $\mathcal{A}$, we can construct a PRG adversary $\mathcal{B}$ s.t.

$$SSadv[\mathcal{A}, \mathcal{E}] \leq 2 \cdot PRGadv[\mathcal{B}, G].$$

Proof: idea

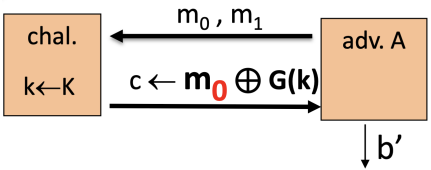
Game 0



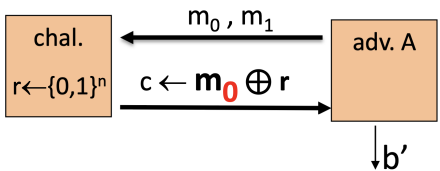


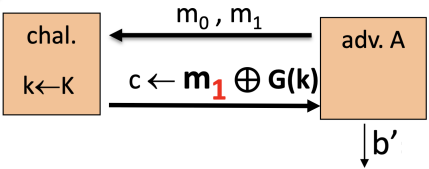
Proof: idea

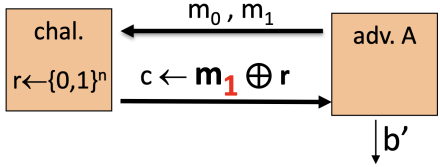
Game 0



Game 1

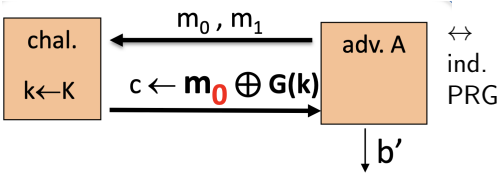




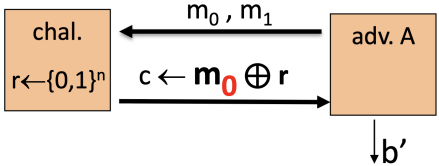


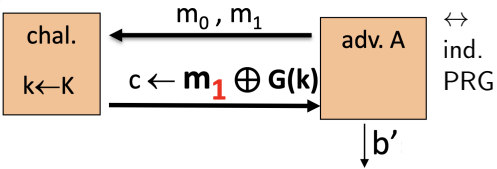
Proof: idea

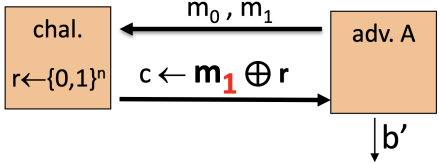
Game 0



Game 1

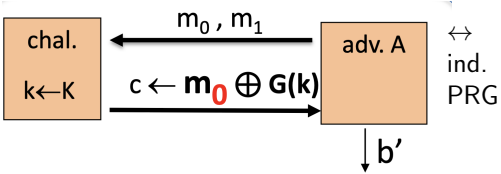




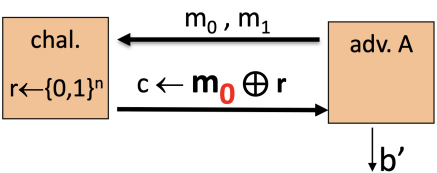


Proof: idea

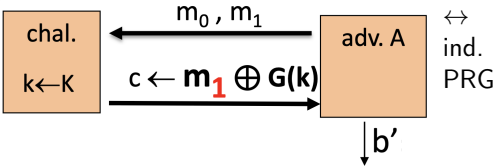
Game 0

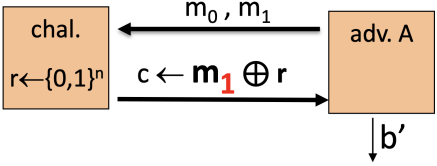


Game 1

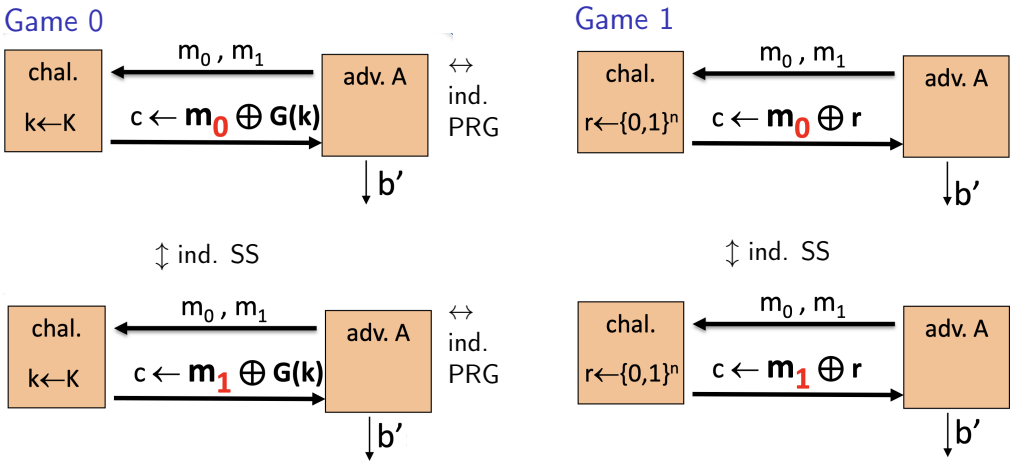


↕ ind. SS





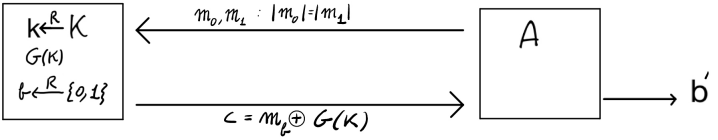
Proof: idea


Game 0
chal.
k←K
m0 , m1
c ← m0 ⊕ G(k)
adv. A
b'
↔
ind.
PRG
ind. SS
chal.
k←K
m0 , m1
c ← m1 ⊕ G(k)
adv. A
b'
↔
ind.
PRG
Game 1
chal.
r←{0,1}n
m0 , m1
c ← m0 ⊕ r
adv. A
b'
ind. SS
chal.
r←{0,1}n
m0 , m1
c ← m1 ⊕ r
adv. A
b'

Proof

Let $\mathcal{A}$ adv. for SS

bit-guessing Game 0:

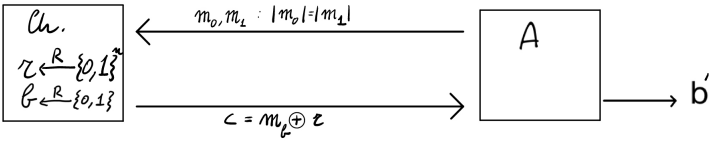


p_{WIN0} = probability $\mathcal{A}$ win in Game 0 $\Rightarrow SSadv^*[\mathcal{A}, \mathcal{E}] = |p_{WIN0} - 1/2|$;

Proof

Let $\mathcal{A}$ adv. for SS

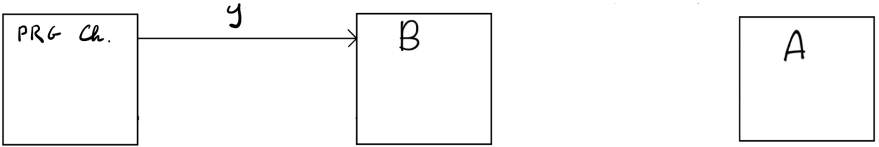
bit-guessing Game 1:



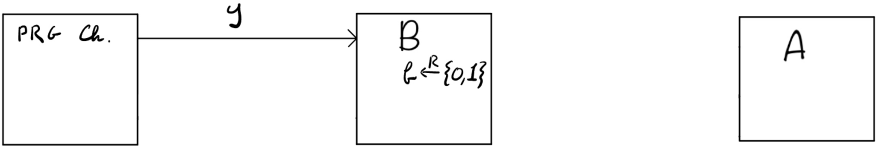
p_{WIN0} = probability $\mathcal{A}$ win in Game 0 $\Rightarrow SSadv^*[\mathcal{A}, \mathcal{E}] = |p_{WIN0} - 1/2|$;

p_{WIN1} = probability $\mathcal{A}$ win in Game 1 $\Rightarrow SSadv^*[\mathcal{A}, OTP] = |p_{WIN1} - 1/2| = 0$

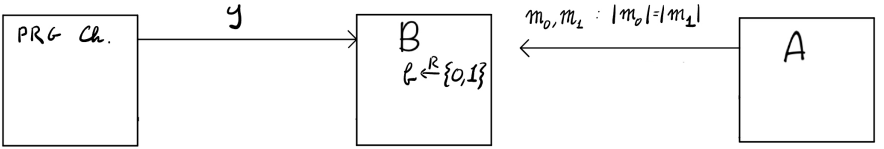
We can construct $\mathcal{B}$ adv. for PRG game as follow:



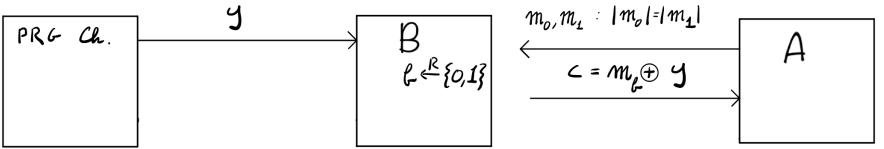
We can construct $\mathcal{B}$ adv. for PRG game as follow:



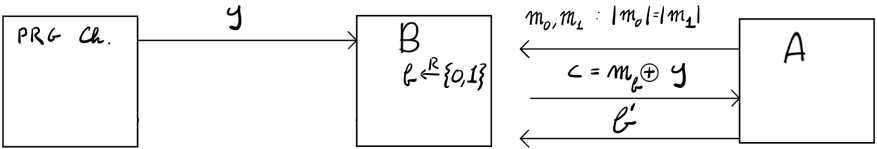
We can construct $\mathcal{B}$ adv. for PRG game as follow:



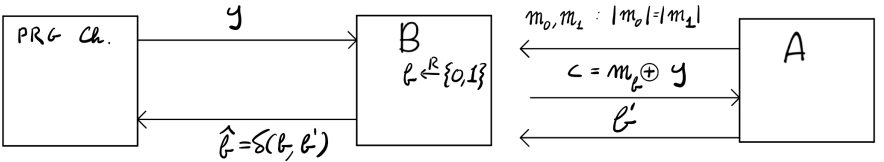
We can construct $\mathcal{B}$ adv. for PRG game as follow:



We can construct $\mathcal{B}$ adv. for PRG game as follow:

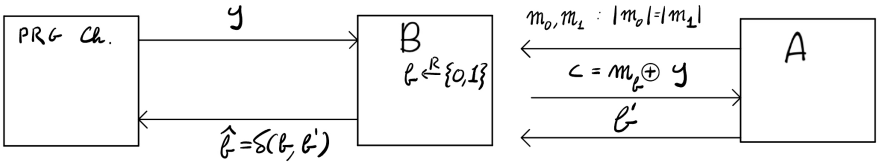


We can construct $\mathcal{B}$ adv. for PRG game as follow:



$\delta(b, b') = 1$ if $b = b'$, 0 if $b \neq b'$.

We can construct $\mathcal{B}$ adv. for PRG game as follow:



$\delta(b, b') = 1$ if $b = b'$, 0 if $b \neq b'$.

1) $Pr[\hat{b} = 1 | \text{Exp. 0 of PRG}] = p_{WIN0}$

2) $Pr[\hat{b} = 1 | \text{Exp. 1 of PRG}] = p_{WIN1} = 1/2$

$\Rightarrow PRGadv[\mathcal{B}, G] = |p_{WIN0} - p_{WIN1}| = |p_{WIN0} - 1/2| = SSadv^*[\mathcal{A}, \mathcal{E}]$

Security issue with one-time pad: two-time pad

Note: a stream cipher key should never be used to encrypt more than one message.

Suppose we encrypt m_1, m_2 with the same key s

$$c_1 = G(s) \oplus m_1,\ c_2 = G(s) \oplus m_2.$$

Then an adversary can compute

$$\Delta = c_1 \oplus c_2 = m_1 \oplus m_2.$$

Two texts contains enough redundancy that given $\Delta = m_1 \oplus m_2$ the adversary can recover both m_1 and m_2 in the clear (or parts of them).

One-time pad is malleable

Semantic security ensures that an adversary cannot read the plaintext, it provides no guarantees for integrity.

- Eve change a ciphertext $c = G(s) \oplus m$ into $c' = c \oplus \Delta$.
- the receiver will decode c' obtaining $c' \oplus G(s) = m \oplus \Delta$
- Knowing neither m nor s, Eve was able to cause the decrypted message to become $m \oplus \Delta$ for Δ of its choice
- Stream-ciphers are said to be **malleable** since an attacker can cause predictable changes to the plaintext

Pros & Cons of stream ciphers

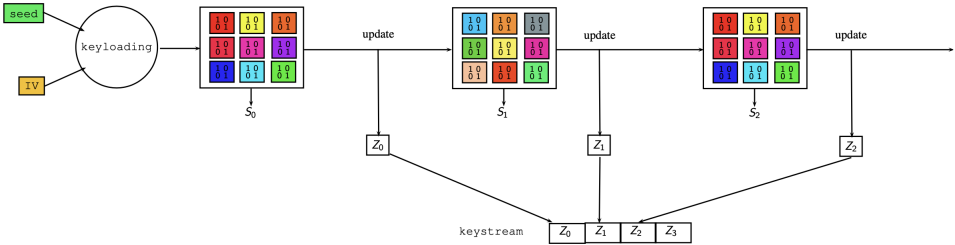
- They do not give rise to error propagation as each bit in the ciphertext depends on just one bit in the plaintext
  1 bit transmission errors will result in 1 bit error in the plaintext
- They are very fast making them ideal for real time applications (e.g., mobile communication services)
- They are easy to implement (in hardware) and they do not require large memory capabilities
- Since stream ciphers process data bitwise, it is crucial that sender and receiver keep their keystreams in perfect synchronisation
  1 bit data loss may have catastrophic consequences as decryptions are performed on the wrong bits after the receiver is out of synch of the sender

How to design a stream cipher?

Main components

- **states** = vector of bits organized in registers
- **key loading** = function that takes the seed (secret) and a (public) Initialization Vector (IV) to compute the initial state for the update function
- **update function**= function mapping a state to the next state (clock function)
- **output function**= function extracting a bit from a state (to compute keystream)

Putting components together



- If Eve is able to recover the states, then it breaks the cipher
- A good stream cipher does not allow Eve to find the states by knowing part of the keystream

Warm up

- The first output bits strongly depend on the initial state
- Hence, it could be easy to recover the initial state by knowing
- To avoid potential problems, it is customary to run a warm-up phase before starting encryption

  applying the update function several times without outputting any bits of keystream
- The warm-up phase allows for avoiding several practical attacks (e.g., RC4)
- It is a highly recommended security best practice

Remarks

- Given any initial state, the states are periodic, since they are in a finite number and at some point we will obtain again one of the previous states
- The keystream is also periodic... this is impossible to avoid
- The smallest number i such that $update^i(S) = S$ is called **period of the keystream**.
- Requirement: the period of the keystream shall be quite large, regardless of the initial state
- This can be achieved by a suitable design of the update function

Linear Feedback Shift Registers

LFSR: How to build "good" update functions

Definition

A linear feedback shift register of length n is a shift register composed by n bits. At any clock the following operations are executed:

- the last bit on the right is output and forms part of the keystream
- the other bits in the register are shifted to the right by one position
- the XOR of some bits of the register are put in the first position on the left.

An n-bit LFSR is defined by a set of integers $V := \{v_1, ..., v_d\}$ where each v_i is in the range $\{0, ..., n-1\}$. The elements of V are called tap positions. An LFSR gives a PRG as follows:

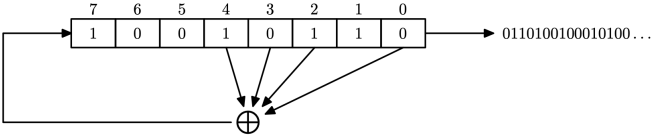
Input: $s = (b_{n-1}, ..., b_0) \in \{0,1\}^n$, $s \neq 0$

Output: $y \in \{0,1\}^L$, $L > n$.

for $i = 1..L$ do

- output b_0
- $b \leftarrow b_{v_1} \oplus ... \oplus b_{v_d}$
- $s \leftarrow (b, b_{n-1}, ..., b_1)$

LFSR: example



- There are only 2^n possible states
- the linear feedback function has the form

$$c_0 b_0 \oplus \ldots \oplus c_{n-1} b_{n-1},$$

  where $c_i \in \{0, 1\}$.
- If the initial state is the vector consisting of all 0s, then the machine will remain in that state forever, so we exclude it
- Hence, the maximum number of steps before the LFSR returns to its initial state is $2^n - 1$
- Given a seed s, the period of s is the number of steps the LFSR takes to return to s
- The period of the LFSR is the maximum period achieved for any seed

- If the period of s is $2^n - 1$, then the LFSR must visit every non-zero state, and so the period for any seed must be $2^n - 1$; we call such a LFSR maximal
- For every n, a maximal LFSR exists
- Of the $2^n - 1$ possible LFSRs, which taps correspond to maximal LFSRs?

Finite Fields

The simplest Galois field is $GF(2)$:

- it contains two elements: 0 and 1
- the binary operations are addition and multiplication both modulo 2

On top of $GF(2)$, it is possible to construct

- $GF(2)[x]$ = the set of polynomials in x with only the elements of $GF(2)$ allowed as coefficients
- $GF(2)[x]/p$ = the quotient of the set of polynomials by p

Example

Consider the polynomial $p(x) = x^3 + x + 1$. In this case, $GF(2)[x]/p$ contains all possible polynomials of degree less than 3, namely

$$\begin{array}{cccc} 0 & 1 & x & x^2 \\ 1+x & 1+x^2 & x+x^2 & 1+x+x^2 \end{array}$$

If we try to use the monomial x as generator, we obtain

$$
\begin{aligned}
x^0 &= 1 & x^4 &= x(x^3) = x(x+1) = x^2 + x \\
x^1 &= x & x^5 &= x^3 + x^2 - p(x) = x^2 + x + 1 \\
x^2 &= x^2 & x^6 &= x^3 + x^2 + x - p(x) = x^2 + 1 \\
x^3 &= x^3 - p(x) = x + 1
\end{aligned}
$$

Except for 0, we have covered all elements in the group

This implies that we can express all of the nonzero elements as powers of x

Note: Making x a generator depends on the polynomial p that is chosen

Polynomials $p(x)$ such that x is a generator of $GF(2)[x]/p(x)$ are called **primitive polynomials**

LFSR and Finite Fields

Primitive polynomials $p(x)$ allow us to design maximal LFSRs by exploiting the following correspondence between these and $GF(2)[x]/p$

- State of an LFSR $\leftrightarrow$ Nonzero element of $GF(2)[x]/p$
- Period of an LFSR $\leftrightarrow$ Order of $GF(2)[x]/p$
- state update function (shift left, xor with taps) $\leftrightarrow$ generation procedure (multiply by x and subtract $p(x)$ when necessary)
- State of an LFSR $\leftrightarrow$ element x^k
- Taps $\leftrightarrow$ Nonzero coefficients of $p(x)$

k	LFSR coefficients 10011	$GF(2)[x]/p(x)$, $p(x) = x^4 + x + 1$
0	$S[0] = 0001$	$x^0 = 0 + 0 + 0 + 1$
1	$S[1] = 0010$	$x^1 = 0 + 0 + x + 0$
2	$S[2] = 0100$	$x^2 = 0 + x^2 + 0 + 0$
3	$S[3] = 1000$	$x^3 = x^3 + 0 + 0 + 0$
4	$S[4] = 0011$	$x^4 = 0 + 0 + x + 1$
5	$S[5] = 0110$	$x^5 = 0 + x^2 + x + 0$
6	$S[6] = 1100$	$x^6 = x^3 + x^2 + 0 + 0$
7	$S[7] = 1011$	$x^7 = x^3 + 0 + x + 1$
8	$S[8] = 0101$	$x^8 = 0 + x^2 + 0 + 1$
9	$S[9] = 1010$	$x^9 = x^3 + 0 + x + 0$
10	$S[10] = 0111$	$x^{10} = 0 + x^2 + x + 1$
11	$S[11] = 1110$	$x^{11} = x^3 + x^2 + x + 0$
12	$S[12] = 1111$	$x^{12} = x^3 + x^2 + x + 1$
13	$S[13] = 1101$	$x^{13} = x^3 + x^2 + 0 + 1$
14	$S[14] = 1001$	$x^{14} = x^3 + 0 + 0 + 1$
15	$S[15] = 0001$	$x^{15} = 0 + 0 + 0 + 1$

Statistical properties of LFSRs

Given a maximum-length LFSR of n bits and reading $2^n - 1$ consecutive bits of the m-sequence that it produces, we have that:

1. one half of the bits are 1 and one half are 0 (actually the 1's are one more than the 0's)
2. there are 2^{n-1} runs:
   - 1/2 of the runs has length 1
   - 1/4 of the runs has length 2
   - ...
   - $1/2^i$ of the runs has length i (for$2 \leq i \leq n-2$)
   - there is only one run of $n-1$ zeros and none of the runs has $n-1$ ones
   - there is only one run of n ones and none of the runs has n zeros.

Example: the sequence 000110111001 is composed by 6 runs of length 3, 2, 1, 3, 2, 1, respectively.

LFSRs main weakness

- LFSRs cannot be used directly in cryptography because of their linearity
- If we intercept $2n$ consecutive bits of the keystream we can reconstruct the feedback polynomial
- Once we obtain the feedback polynomial, we are able to reproduce the whole keystream from any (short) subsequence of the keystream having length n

Hence LFSRs are vulnerable with respect to known-plaintext attacks and they must not be used as keystream generators

- Although they can be used as components!
- In particular when iterated and combined by using some kind of **non-linear functions**